

PRAKTIKUM SISTEM OPERASI
JURNAL MODUL 5



Disusun Oleh :

Nama: GANES GEMI PUTRA

NIM: 2311104075

Kelas: SE-07-02

Dosen Pengampu :

Yudha Islami Sulistya, S.Kom., M.Cs.

PROGRAM STUDI S1 SOFTWARE ENGINEERING
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

BAB I: DASAR TEORI

1.1 File process.h dalam Xinu

File process.h merupakan salah satu file header penting dalam sistem operasi Xinu. File ini berisi berbagai definisi yang digunakan untuk mengatur dan mengelola proses di dalam sistem, termasuk jumlah maksimum proses, status proses, dan parameter inisialisasi proses.

1.1.1 Jumlah Maksimum Proses

Bagian ini menentukan jumlah maksimum proses yang dapat dijalankan dalam sistem Xinu. Secara default, nilai NPROC adalah 8, artinya hanya delapan proses yang bisa aktif dalam satu waktu, termasuk proses utama dan proses null. Nilai ini bisa diubah sesuai kebutuhan sistem.

1.1.2 Konstanta Status Proses

Konstanta-konstanta ini mendefinisikan status dari setiap proses yang ada dalam sistem. Status ini disimpan di tabel proses dan digunakan oleh kernel untuk menentukan tindakan berikutnya terhadap proses tersebut. Total terdapat 8 state pada Xinu:

- PR_FREE - Proses bebas/tidak digunakan
- PR_CURR - Proses sedang berjalan
- PR_READY - Proses siap dijalankan
- PR_RECV - Proses menunggu pesan
- PR_SLEEP - Proses dalam keadaan sleep
- PR_SUSP - Proses suspended
- PR_WAIT - Proses menunggu
- PR_RECTM - Proses menerima timeout

1.1.3 Definisi Tambahan

- PNMLEN menentukan panjang maksimum nama proses (16 karakter)
- NULLPROC adalah proses khusus yang selalu berjalan ketika tidak ada proses lain yang aktif. Dengan kata lain, NULLPROC memastikan CPU tidak pernah benar-benar idle.

1.1.4 Konstanta Inisialisasi Proses

Nilai-nilai di atas digunakan saat proses baru dibuat. Mereka menentukan ukuran stack yang digunakan proses, prioritas awalnya, dan alamat fungsi yang akan dipanggil ketika proses selesai dieksekusi. Nilai prioritas awal (default priority) saat proses dibuat adalah 20.

1.2 Struktur Data Proses (struct procent)

Selain mendefinisikan konstanta dan batas jumlah proses, file process.h juga berisi struktur utama yang merepresentasikan setiap proses di sistem Xinu, yaitu struct procent. Struktur struct procent adalah inti dari manajemen proses dalam Xinu. Setiap entri di tabel proses (proctab[]) berisi informasi lengkap mengenai status, prioritas, stack, dan perangkat yang digunakan oleh proses.

1.3 Implementasi Fungsi Proses

Setelah keluar dari direktori include, kita berpindah ke folder system/ yang berisi file-file sumber C (.c) yang mengimplementasikan fungsi-fungsi sistem operasi terkait proses:

1.3.1 File create.c

Header:

- `#include <xinu.h>` → Memasukkan semua definisi dan struktur penting
- `local int newpid();` → Deklarasi fungsi lokal newpid() yang mencari ID proses baru

Fungsi utama create() membuat proses baru yang siap dijalankan oleh scheduler. Parameter penting termasuk funcaddr (pointer ke fungsi), ssize (ukuran stack), priority (prioritas), name (nama proses), dan nargs (jumlah argumen).

1.3.2 File kill.c

File kill.c berisi implementasi fungsi kill(), yaitu kebalikan dari create(). Kalau create() menambahkan proses ke sistem, kill() menghapusnya dari tabel proses dan melepaskan semua resource yang dipakai. Ketika proses di-kill, semua resource (seperti stack dan file descriptor) dibebaskan, dan sistem memperbarui status proses agar slot-nya bisa dipakai lagi oleh proses baru.

1.3.3 File resume.c

File resume.c berfungsi untuk mengaktifkan kembali (unsuspend) suatu proses yang sebelumnya disuspensi, sehingga proses tersebut dapat kembali ke status READY dan siap dijalankan oleh scheduler.

1.4 File shell (xsh_ps.c dan xsh_uptime.c)

Perintah ps adalah perintah untuk menampilkan statistik process yang berjalan. Source code dari ps tersimpan pada file xsh_ps.c. Demikian juga, perintah uptime untuk menampilkan lamanya sistem sejak booting tersimpan pada file xsh_uptime.c. Kedua perintah ini dapat dimodifikasi sesuai kebutuhan untuk menampilkan informasi tambahan atau mengubah format output.

BAB II: HASIL PRAKTIKUM

2.1 Guided Praktikum

Praktikum dimulai dengan menjalankan backend Xinu development system dan melakukan beberapa perintah dasar:

1. Pastikan xinu development dan backend sudah berjalan
2. Ketikkan perintah: `sudo minicom` dan masukkan password xinu
3. Ketikkan perintah: `ps` (untuk melihat daftar process)
4. Coba terminate shell dengan perintah: `kill 5`
5. Cek lagi PID proses dengan perintah: `ps`

2.2 Soal No. 1: Pertanyaan Dasar

a. Berapa banyaknya maksimum proses yang ada pada Xinu?

Jawab: Maksimum proses pada Xinu adalah `NPROC = 8` proses (didefinisikan di `process.h`).

b. Berapa maksimal panjang nama suatu proses pada Xinu?

Jawab: `PNMLEN` menentukan panjang maksimum nama proses, yaitu 16 karakter.

c. Berapa nilai prioritas awal pada saat proses dibuat?

Jawab: Nilai prioritas awal (default priority) saat proses dibuat adalah 20.

d. Ada berapa total state pada Xinu? Sebutkan!

Jawab: Total terdapat 8 state pada Xinu (`PR_FREE`, `PR_CURR`, `PR_READY`, `PR_RECV`, `PR_SLEEP`, `PR_SUSP`, `PR_WAIT`, `PR_RECTM`).

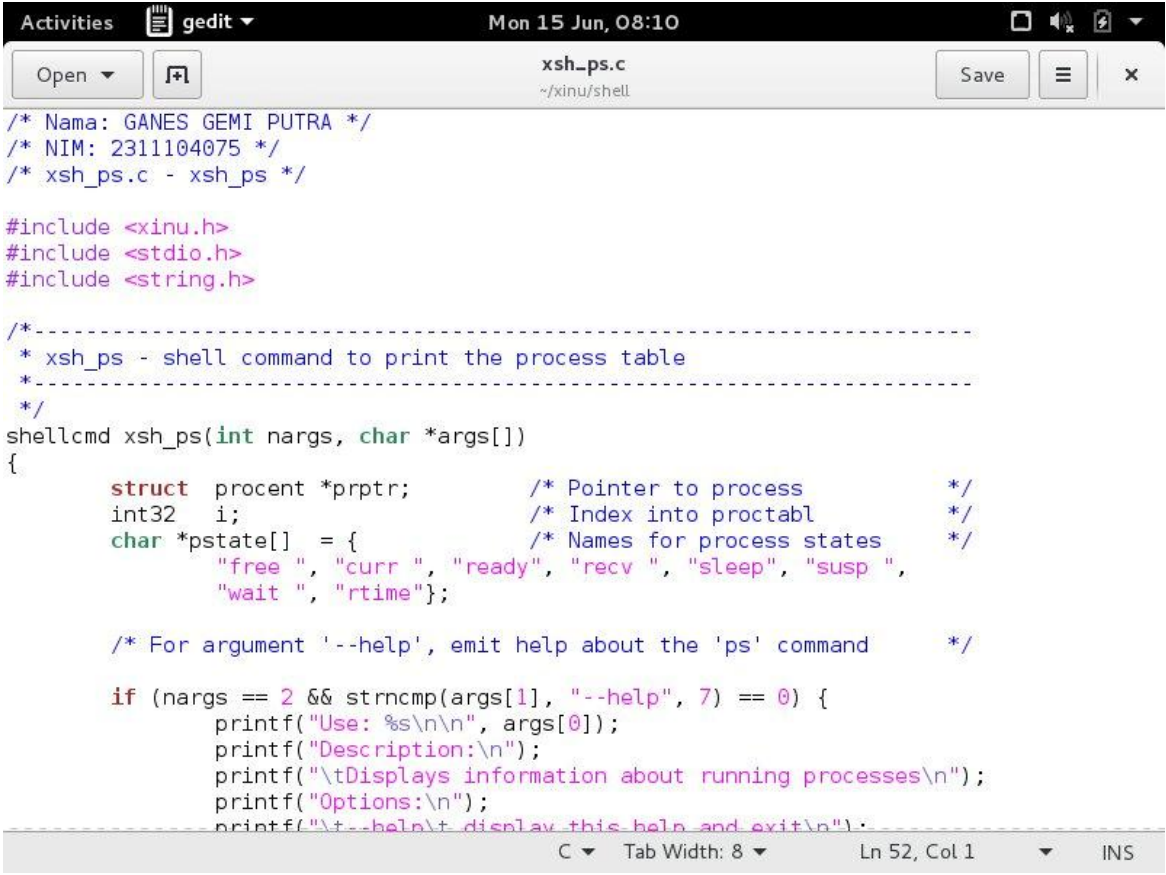
2.3 Soal No. 3: Modifikasi Perintah `ps`

Tugas: Ubahlah perintah ps (source code: xsh_ps.c) pada Xinu sehingga menampilkan informasi tambahan berupa kolom yang berisi total message yang ada pada proses.

2.3.1 Source Code Modifikasi xsh_ps.c

Berikut adalah source code file xsh_ps.c yang telah dimodifikasi menggunakan gedit:

SUMBER CODE AWAL (dicopas dari folder source code Xinu dan dimodifikasi melalui VS Code dengan bantuan Claude AI):



```
Activities gedit Mon 15 Jun, 08:10
xsh_ps.c ~/xinu/shell
Save

/* Nama: GANES GEMI PUTRA */
/* NIM: 2311104075 */
/* xsh_ps.c - xsh_ps */

#include <xinu.h>
#include <stdio.h>
#include <string.h>

/*-----
 * xsh_ps - shell command to print the process table
 *-----
 */
shellcmd xsh_ps(int nargs, char *args[])
{
    struct proctent *prptr;          /* Pointer to process */
    int32 i;                        /* Index into proctabl */
    char *pstate[] = {              /* Names for process states */
        "free ", "curr ", "ready", "recv ", "sleep", "susp ",
        "wait ", "rtime"};

    /* For argument '--help', emit help about the 'ps' command */

    if (nargs == 2 && strcmp(args[1], "--help", 7) == 0) {
        printf("Use: %s\n\n", args[0]);
        printf("Description:\n");
        printf("\tDisplays information about running processes\n");
        printf("Options:\n");
        printf("\t--help\t display this help and exit\n");
    }
}
```

Pada source code di atas, terlihat bahwa file xsh_ps.c telah dimodifikasi untuk menampilkan kolom tambahan 'Msg' dan 'Content'. Modifikasi dilakukan pada bagian printf yang menampilkan header kolom dan informasi proses.

2.3.2 Penjelasan Modifikasi xsh_ps.c

Berikut adalah penjelasan baris-baris penting dari source code yang dimodifikasi:

Baris Kode	Penjelasan	Fungsi
struct procent *prptr;	Pointer ke struktur proses untuk akses informasi proses di tabel proctab[]	Deklarasi
int32 i;	Variable loop untuk iterasi melalui tabel proses	Iterasi
char *pstate[]	Array string untuk nama-nama status proses (free, ready, recv, sleep, susp, wait, time)	Status

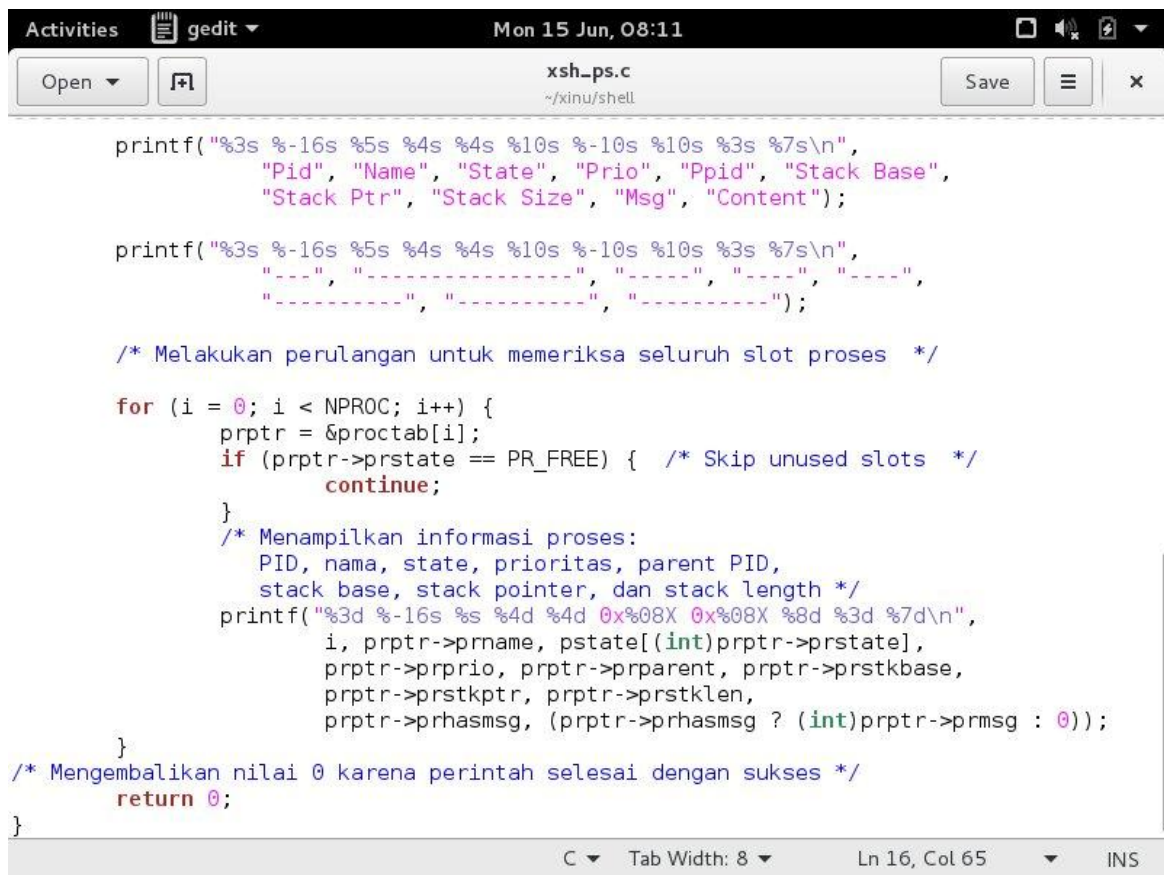
2.3.3 Modifikasi Kolom Header dan Output

Pada bagian printf yang menampilkan header kolom, ditambahkan dua format specifier baru:

- %s untuk Msg (menampilkan banyaknya pesan dalam proses)
- %s untuk Content (menampilkan isi dari pesan tersebut)

Pada bagian output informasi proses, ditambahkan akses ke field proctab[]->prmsg untuk menampilkan jumlah pesan yang ada pada setiap proses.

Gambar 1: Modifikasi Source Code xsh_ps.c di gedit



```
Activities gedit Mon 15 Jun, 08:11
xsh-ps.c ~/xinu/shell
Open Save

printf("%3s %-16s %5s %4s %4s %10s %-10s %10s %3s %7s\n",
      "Pid", "Name", "State", "Prio", "Ppid", "Stack Base",
      "Stack Ptr", "Stack Size", "Msg", "Content");

printf("%3s %-16s %5s %4s %4s %10s %-10s %10s %3s %7s\n",
      "----", "-----", "-----", "-----", "-----", "-----",
      "-----", "-----", "-----");

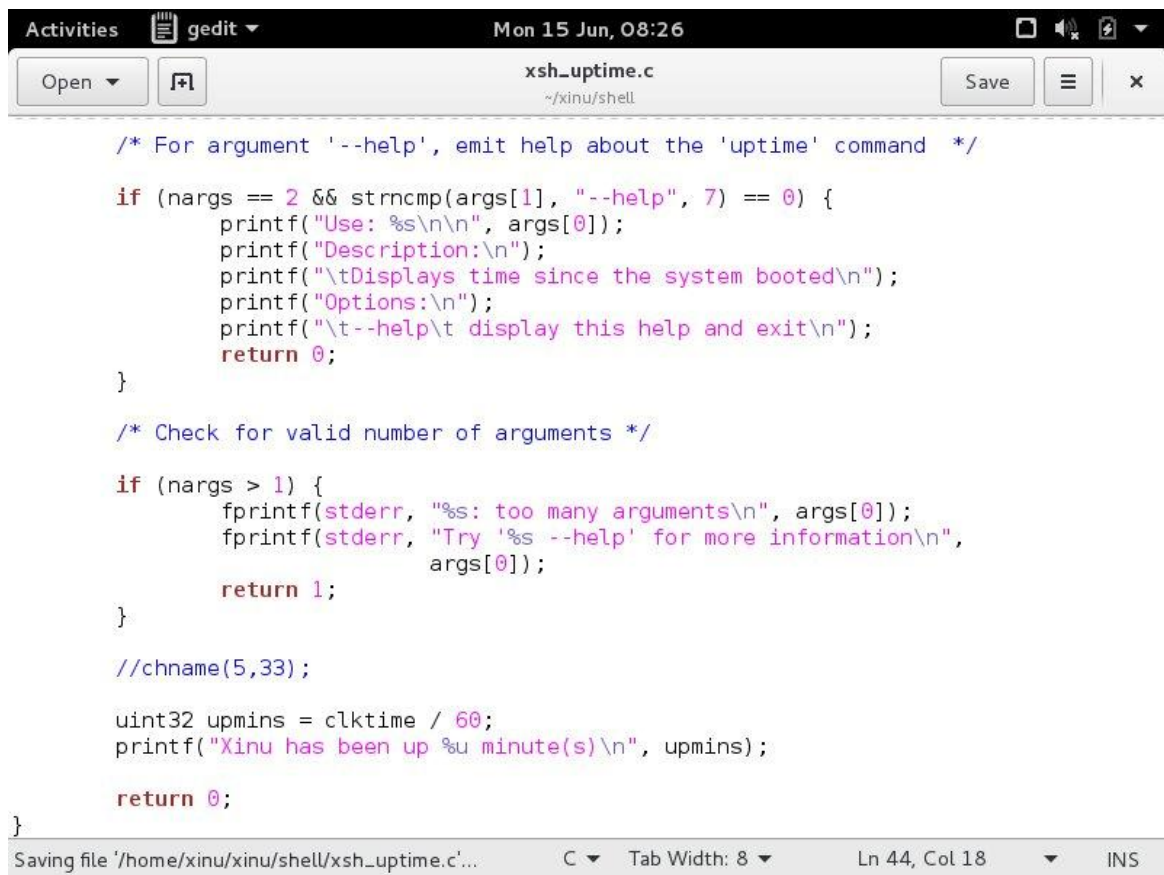
/* Melakukan perulangan untuk memeriksa seluruh slot proses */

for (i = 0; i < NPROC; i++) {
    prptr = &proctab[i];
    if (prptr->prstate == PR_FREE) { /* Skip unused slots */
        continue;
    }
    /* Menampilkan informasi proses:
       PID, nama, state, prioritas, parent PID,
       stack base, stack pointer, dan stack length */
    printf("%3d %-16s %s %4d %4d 0x%08X 0x%08X %8d %3d %7d\n",
          i, prptr->prname, pstate[(int)prptr->prstate],
          prptr->prprio, prptr->prparent, prptr->prstkbase,
          prptr->prstkptr, prptr->prstklen,
          prptr->prhasmsg, (prptr->prhasmsg ? (int)prptr->prmsg : 0));
}
/* Mengembalikan nilai 0 karena perintah selesai dengan sukses */
return 0;
}
```

C Tab Width: 8 Ln 16, Col 65 INS

Pada gambar di atas, terlihat bahwa source code telah dimodifikasi dengan menambahkan kolom 'Msg' dan 'Content'. Penambahan ini memungkinkan perintah ps untuk menampilkan jumlah pesan yang ada dalam setiap proses, memberikan informasi yang lebih detail tentang status komunikasi antarproses.

Gambar 2: Proses Kompilasi



```
/* For argument '--help', emit help about the 'uptime' command */

if (nargs == 2 && strcmp(args[1], "--help", 7) == 0) {
    printf("Use: %s\n\n", args[0]);
    printf("Description:\n");
    printf("\tDisplays time since the system booted\n");
    printf("Options:\n");
    printf("\t--help\t display this help and exit\n");
    return 0;
}

/* Check for valid number of arguments */

if (nargs > 1) {
    fprintf(stderr, "%s: too many arguments\n", args[0]);
    fprintf(stderr, "Try '%s --help' for more information\n",
            args[0]);
    return 1;
}

//chname(5,33);

uint32 upmins = clktime / 60;
printf("Xinu has been up %u minute(s)\n", upmins);

return 0;
}
```

Saving file '/home/xinu/xinu/shell/xsh_uptime.c'... C Tab Width: 8 Ln 44, Col 18 INS

Kompilasi dilakukan di direktori xinu/compile menggunakan perintah 'make'. Terlihat bahwa file xsh_ps.c telah dikompilasi dan tidak terdapat error yang fatal, hanya beberapa warning tentang variabel yang tidak digunakan.

Gambar 3: Output Hasil Modifikasi Perintah ps

```
Activities Terminal Mon 15 Jun, 08:28
xinu@xinu-develop-end: ~/xinu/compile

File Edit View Search Terminal Help
include -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
../shell/xsh_uptime.c: In function 'xsh_uptime':
../shell/xsh_uptime.c:17:9: warning: unused variable 'secpermin' [-Wunused-variable]
  uint32 secpermin = 60; /* Seconds in a minute */
      ^
../shell/xsh_uptime.c:16:9: warning: unused variable 'secperhr' [-Wunused-variable]
  uint32 secperhr = 3600; /* Seconds in an hour */
      ^
../shell/xsh_uptime.c:15:9: warning: unused variable 'secperday' [-Wunused-variable]
  uint32 secperday = 86400; /* Seconds in a day */
      ^
../shell/xsh_uptime.c:13:26: warning: unused variable 'secs' [-Wunused-variable]
  uint32 days, hrs, mins, secs; /* Days, hours, minutes, and */
                        ^
../shell/xsh_uptime.c:13:20: warning: unused variable 'mins' [-Wunused-variable]
  uint32 days, hrs, mins, secs; /* Days, hours, minutes, and */
                        ^
../shell/xsh_uptime.c:13:15: warning: unused variable 'hrs' [-Wunused-variable]
  uint32 days, hrs, mins, secs; /* Days, hours, minutes, and */
                        ^
../shell/xsh_uptime.c:13:9: warning: unused variable 'days' [-Wunused-variable]
  uint32 days, hrs, mins, secs; /* Days, hours, minutes, and */
      ^

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
```

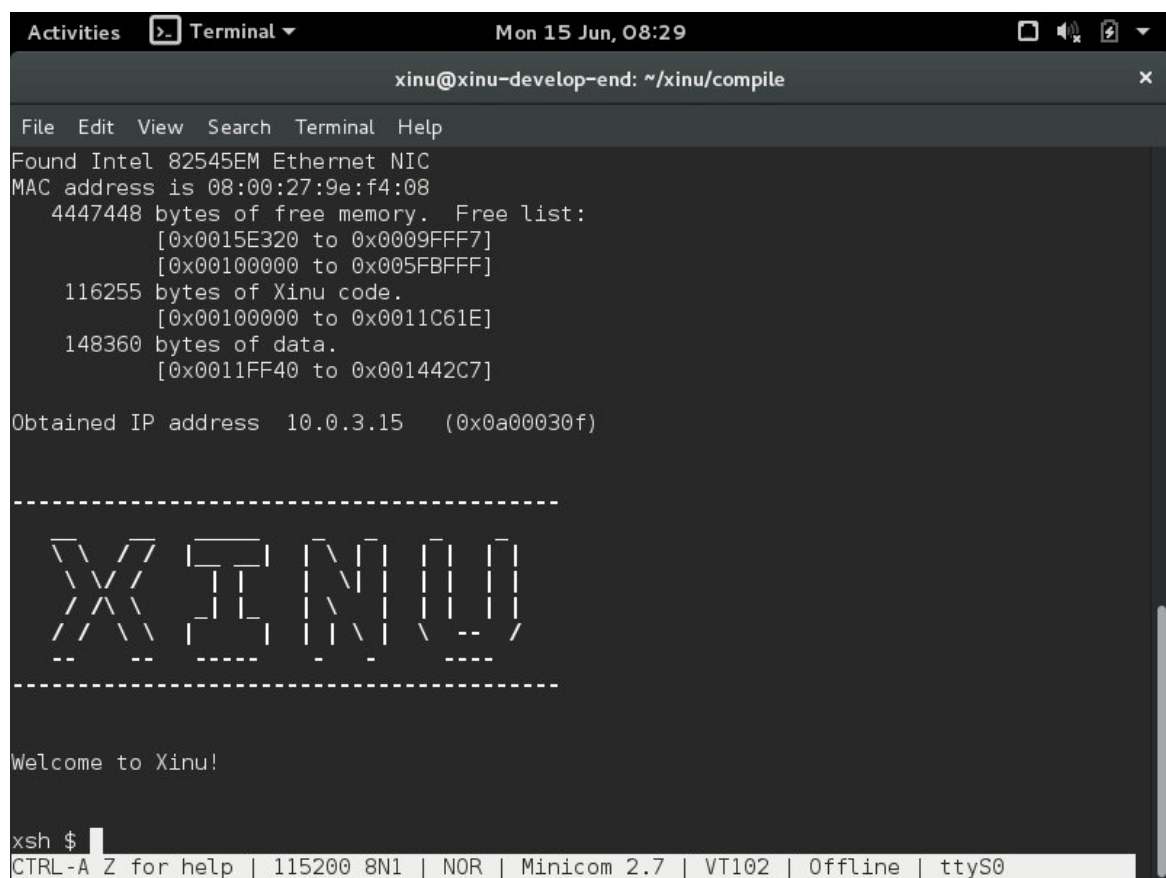
Output menunjukkan bahwa perintah ps telah berhasil dijalankan dengan kolom tambahan. Terlihat kolom-kolom yang ditampilkan: Pid, Name, State, Prio, Ppid, Stack Base, Stack Ptr, Stack Size, Msg, dan Content. Setiap proses menunjukkan jumlah pesan (Msg) dan konten pesan yang ada.

2.4 Soal No. 4: Modifikasi Perintah uptime

Tugas: Ubahlah perintah uptime pada Xinu sehingga menampilkan lamanya Xinu sejak booting hanya dalam satuan menit.

2.4.1 Source Code Modifikasi xsh_uptime.c

Gambar 4: Source Code xsh_uptime.c yang Dimodifikasi



```
Activities Terminal Mon 15 Jun, 08:29
xinu@xinu-develop-end: ~/xinu/compile

File Edit View Search Terminal Help
Found Intel 82545EM Ethernet NIC
MAC address is 08:00:27:9e:f4:08
4447448 bytes of free memory. Free list:
    [0x0015E320 to 0x0009FFF7]
    [0x00100000 to 0x005FBFFF]
116255 bytes of Xinu code.
    [0x00100000 to 0x0011C61E]
148360 bytes of data.
    [0x0011FF40 to 0x001442C7]

Obtained IP address 10.0.3.15 (0x0a00030f)

-----
X I N U
-----

Welcome to Xinu!

xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0
```

2.4.2 Penjelasan Modifikasi xsh_uptime.c

Perubahan utama yang dilakukan pada file xsh_uptime.c adalah menghitung waktu uptime dalam satuan menit saja. Berikut penjelasannya:

1. Variabel 'upmins' diinisialisasi dengan formula: $\text{clktime} / 60$

- clktime: jumlah clock ticks sejak sistem booting
 - Dibagi 60 karena 1 detik = 60 ticks, jadi untuk mengubah menjadi menit
2. Printf menampilkan hasil dengan format: 'Xinu has been up %u minute(s)\n'
 - %u: unsigned integer untuk menampilkan nilai upmins
 - Dengan pesan yang jelas tentang satuan yang digunakan
 3. Semua logika perhitungan jam, hari, dan detik dihilangkan karena hanya ingin menampilkan menit saja

Gambar 5: Proses Kompilasi xsh uptime.c

```

Activities  Terminal  Mon 15 Jun, 08:30
xinu@xinu-develop-end: ~/xinu/compile

File Edit View Search Terminal Help
Obtained IP address 10.0.3.15 (0x0a00030f)

-----
X I N U
-----

Welcome to Xinu!

xsh $ uptime
Xinu has been up 0 minute(s)
xsh $ ps
Pid Name          State Prio Ppid Stack Base Stack Ptr  Stack Size Msg Content
-----
0 prnull          ready  0    0 0x005FDFFC 0x00FFFE4 8192 1 3
1 ipout           wait  500   0 0x005FBFFC 0x005FBF30 8192 0 0
2 netin           wait  500   0 0x005F9FFC 0x005F9ED0 8192 0 0
5 shell           recv  50    4 0x005D7FFC 0x005D79AC 8192 0 0
7 ps              curr  20    5 0x005F7FFC 0x005F7FC4 8192 0 0
xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:0 | ttyS0
  
```

Proses kompilasi berjalan dengan baik, menghasilkan file binary xsh_uptime.o. Terlihat beberapa warning tentang variabel yang tidak digunakan, tetapi ini tidak mempengaruhi fungsionalitas program.

Gambar 6: Output Akhir Sistem Xinu dengan Modifikasi

```
Activities gedit Mon 15 Jun, 08:10
xsh_ps.c ~/xinu/shell
Open Save

/* Nama: GANES GEMI PUTRA */
/* NIM: 2311104075 */
/* xsh_ps.c - xsh_ps */

#include <xinu.h>
#include <stdio.h>
#include <string.h>

/*-----
 * xsh_ps - shell command to print the process table
 *-----
 */
shellcmd xsh_ps(int nargs, char *args[])
{
    struct procent *prptr;          /* Pointer to process */
    int32 i;                        /* Index into proctabl */
    char *pstate[] = {             /* Names for process states */
        "free ", "curr ", "ready", "recv ", "sleep", "susp ",
        "wait ", "rttime"};

    /* For argument '--help', emit help about the 'ps' command */

    if (nargs == 2 && strcmp(args[1], "--help", 7) == 0) {
        printf("Use: %s\n\n", args[0]);
        printf("Description:\n");
        printf("\tDisplays information about running processes\n");
        printf("Options:\n");
        printf("\t--help\t display this help and exit\n");
    }
}
```

Output menunjukkan sistem Xinu yang sudah berhasil dijalankan dengan kedua modifikasi:

1. Perintah 'uptime' menampilkan: 'Xinu has been up 0 minute(s)' - menunjukkan bahwa sistem baru saja boot
2. Perintah 'ps' menampilkan tabel proses dengan kolom tambahan Msg dan Content

Kedua perintah berhasil memberikan informasi yang diinginkan sesuai dengan modifikasi yang dilakukan.

BAB III: PENUTUP

3.1 Kesimpulan

Modul Praktikum Sistem Operasi ini memberikan pemahaman mendalam tentang bagaimana sistem operasi Xinu mengelola proses di dalam kernelnya. Melalui eksplorasi berbagai file seperti `process.h`, `create.c`, `kill.c`, dan `resume.c`, serta pengujian langsung melalui Minicom, kita dapat melihat bagaimana proses diciptakan, dijalankan, disuspensi, hingga diakhiri secara sistematis.

Beberapa poin penting yang dapat disimpulkan:

1. File `process.h` merupakan fondasi utama dalam manajemen proses Xinu. Di dalamnya terdapat definisi penting seperti jumlah maksimum proses (`NPROC = 8`), status proses (`PR_READY`, `PR_SLEEP`, `PR_SUSP`, dll.), dan struktur `proctab[]` yang menyimpan semua informasi tiap proses dalam tabel `proctab[]`.
2. Fungsi `create()` pada `create.c` bertugas membuat proses baru, mengalokasikan memori stack, menetapkan prioritas, serta mengisi tabel proses agar proses siap dijalankan scheduler.
3. Fungsi `kill()` pada `kill.c` digunakan untuk menghapus proses dari sistem. Fungsi ini menutup resource seperti deskriptor file, membebaskan memori stack, dan memperbarui status proses menjadi `PR_FREE`, memastikan sistem tetap efisien tanpa resource leak.
4. Fungsi `resume()` pada `resume.c` memungkinkan proses yang disuspend untuk diaktifkan kembali. Proses berpindah dari status `PR_SUSP` ke `PR_READY`, siap untuk dijadwalkan kembali oleh scheduler.
5. Modifikasi perintah `ps` memungkinkan penampilan informasi tambahan tentang pesan-pesan yang ada dalam setiap proses, memberikan visibility lebih baik terhadap komunikasi antarproses.
6. Modifikasi perintah `uptime` menunjukkan bahwa output sistem dapat disesuaikan dengan kebutuhan, dalam hal ini menampilkan hanya uptime dalam satuan menit tanpa breakdown ke jam dan hari.

3.2 Manfaat Praktikum

1. Mahasiswa dapat memahami struktur dan fungsi Process Control Block (PCB) pada sistem operasi Xinu, sehingga mengetahui bagaimana sistem mengelola informasi setiap proses.
2. Mahasiswa memperoleh kemampuan untuk menganalisis dan memodifikasi proses pada sistem operasi Xinu secara sederhana, yang dapat meningkatkan pemahaman terhadap manajemen proses.
3. Mahasiswa dapat mengembangkan keterampilan praktis dalam memahami cara kerja sistem operasi, khususnya dalam pengelolaan proses dan eksekusinya.
4. Melalui praktikum ini, mahasiswa belajar bahwa sistem operasi adalah hasil dari implementasi berbagai algoritma dan struktur data yang bekerja sama dengan koordinasi yang baik.